Shivani.S-192471030

66.How can you optimize the N-Queens solution using auxiliary arrays for faster constraint checking?

Ans:

```
def solve_n_queens(n):
 board = [["." for _ in range(n)] for _ in range(n)]

 cols = [False] * n
 diag1 = [False] * (2 * n - 1)
 diag2 = [False] * (2 * n - 1)

 solutions = []

 def backtrack(row):
 if row == n:
 solution = ["".join(r) for r in board]
 solutions.append(solution)
 return

 for col in range(n):
 d1 = row - col + n - 1
 d2 = row + col

 if cols[col] or diag1[d1] or diag2[d2]:
 continue

 board[row][col] = "Q"
 cols[col] = True
 diag1[d1] = True
```

```
 diag2[d2] = True

 backtrack(row + 1)

 board[row][col] = "."
 cols[col] = False
 diag1[d1] = False
 diag2[d2] = False

 backtrack(0)
 return solutions



n = 4
answer = solve_n_queens(n)

print("Number of solutions:", len(answer))

for solution in answer:
 for row in solution:
 print(row)
 print()
```

67.Explain how backtracking can be used to solve a variation of the Subset Sum problem with size constraints.

Ans:

```
def subset_sum_with_size(arr, target, k):
 result = []

 def backtrack(index, current_subset, current_sum):
```

```
if len(current_subset) == k:
if current_sum == target:
result.append(current_subset[:])
return

if index == len(arr):
return

if current_sum > target:
return

# Include current element
current_subset.append(arr[index])
backtrack(index + 1, current_subset, current_sum + arr[index])
current_subset.pop()

# Exclude current element
backtrack(index + 1, current_subset, current_sum)

backtrack(0, [], 0)

return result



arr = [3, 5, 6, 7, 8]
target = 15
k = 2
```

```
answer = subset_sum_with_size(arr, target, k)

print("Subsets with sum", target, "and size", k, ":")

for subset in answer:
 print(subset)
```